

Project Documentation

Author-Branded Book Reader App System

Date: July 2026

1. Project Overview

This project is a book distribution system designed for authors who want to publish and deliver books through dedicated branded mobile applications. Each author gets a separate Flutter app with its own branding, while all authors are managed from one shared admin panel.

The system is not a SaaS platform where many authors use one common mobile app. Instead, each author receives a custom-built app that looks and feels like their own product. The main goal is to give readers a simple, reliable, offline-first reading experience while allowing authors and admins to manage content efficiently from one central panel.

The system supports two book formats:

- **IMAGE books:** chapters are made of ordered page images, with optional audio.
- **TEXTBOOKS:** chapters are written as text content, with optional audio.

The first book of an author can be bundled inside the app, so the reader has immediate access after installation, even without internet. Additional books can be discovered online and downloaded chapter by chapter.

2. Project Goals

The project is designed to achieve the following goals:

1. Give each author a dedicated branded mobile app.
2. Keep the reader experience simple and offline-friendly.
3. Allow authors to manage books, chapters, text, images, and audio from one admin panel.

4. Keep downloaded content available locally using SQLite.
5. Support smart updates so readers receive only changed content when authors edit books.
6. Keep the first version practical, stable, and commercially useful without overcomplicating the system.

3. System Summary

The system has three main parts:

3.1 Mobile Reader App

A Flutter application built from a shared template and customized for each author. The app contains the branding, author ID, package name, app icon, splash screen, and API settings for that author.

3.2 Admin Panel

A single web-based admin panel built with Next.js for the user interface and Node.js or PHP for the backend API layer. The admin panel allows authors and administrators to log in, manage books, upload chapters, publish content, hide content, and update branding settings.

3.3 Backend and Database

A shared backend service connected to a MySQL database. The backend stores author data, books, chapters, media files, version numbers, and update information. Every app request is filtered by `authorId` so each mobile app only receives the correct author's content.

4. System Architecture

The system is built around a shared backend and separate author-branded mobile apps.

4.1 High-Level Flow

- A new author account is created in the admin panel.
- The author receives credentials for the admin panel.
- A Flutter app is generated from the shared template for that author.
- The app is configured with the author's ID and branding values.
- The app connects to the shared backend API.
- The backend returns only data that belongs to that author.
- Readers install the app and access the first bundled book immediately.
- Additional books and updates are downloaded from the backend when internet is available.

4.2 Core Design Principle

The backend is the **single source of truth** for book content, versions, and publishing state.

The mobile app stores downloaded content locally in SQLite for offline use, but the backend controls what is current.

5. User Roles

5.1 Super Admin

A system-level administrator who can manage all authors, access all content, and maintain the platform.

- Can create Author accounts and trigger automated invite emails for onboarding.
- Can suspend or delete Author accounts.
- Can use a "Log in as [Author]" impersonation feature to troubleshoot specific user dashboards.
- Can utilize a one-click "Unpublish" moderation override for any broken or inappropriate content.

5.2 Author Admin

A user account created for each author. This account allows the author to log in to the admin panel and manage only their own books and content.

5.3 Reader

The person who installs the author-branded app and reads or listens to the books.

6. Mobile Reader App Requirements

The mobile app is the main product delivered to readers. It must be simple, fast, and reliable.

6.1 App Branding

Each app is built separately for a specific author and can include:

- App name
- App icon
- Splash screen
- Primary colour theme
- Package name
- Backend URL
- Author ID

6.2 First-Run Experience

When a reader installs the app:

- One selected book is already available in the app.
- The reader can open and read that book without internet.
- If internet is available, the app can also check for additional books and updates.

6.3 Book Types

IMAGE Book

An IMAGE book is made of chapters that contain ordered page images. The reader can swipe through the pages like a visual book. Audio can be attached to the chapter if needed.

Textbook

A TEXTBOOK is made of chapters that contain written text content. The reader scrolls through the text in a clean reading layout. Audio can also be attached to the chapter.

6.4 Book Type Rule

A single book must use only one type:

- IMAGE
- TEXT

A book cannot mix image pages and text pages in the same book.

6.5 Chapter Downloads

Books are downloaded chapter by chapter. This approach keeps downloads smaller and makes updates easier to manage.

The app should support:

- Download one chapter
- Download remaining chapters
- Download the full book
- Resume failed downloads
- Show download progress

6.6 Audio Support

The app should support audio in two ways:

- **Streaming:** play audio directly when internet is available
- **Download:** save audio locally for offline playback

Audio playback should include:

- Play/pause
- Seek
- Resume from last position
- Playback speed from 0.75x to 2x

6.7 Offline Storage

SQLite is used for local storage inside the Flutter app. The app should store:

- Reading progress
- Bookmarks
- Favourites
- Downloaded chapter metadata
- Local settings such as dark mode
- Last opened book and chapter
- Local version numbers for update checks

6.8 Update Behaviour

The app must check for updates when internet is available.

- If only **text** content changes, the app can sync automatically.
- If **images** or **audio** change, the app should notify the reader and ask them to download the update.
- The app should not overwrite downloaded content silently unless the change is text-only and safe to sync.

6.9 Reader Features

The reader app should include:

- Smooth page swiping for image books
- Clean scrolling reader for textbooks
- Continue reading from last position
- Download management
- Dark mode
- Audio player
- Bookmarks
- Favourites

7. Admin Panel Requirements

The admin panel is the control center for authors and platform administrators.

7.1 Login and Access

The Super Admin can create an account for Each author and give their own login credentials. After login, the author can only manage their own books and related content.

7.2 Book Management

The admin panel should allow authors to:

- Create books
- Edit book title and description
- Archive/Restore books utilizing a soft-delete mechanism.
- Toggle a "Live Preview" mode to see how content, images, and audio will render for the end-user.
- Rely on pre-publish validation that flags empty chapters or missing media before a book goes live.
- Utilize a drag-and-drop interface to reorder chapters and content blocks.
- Add cover images
- Select book type: IMAGE or TEXT
- Publish books
- Hide books
- Reorder books
- Set publishing date if scheduling is enabled later

7.3 Chapter Management

The admin panel should allow authors to:

- Create chapters under selected book
- Edit chapter title
- Reorder chapters
- Upload page images for IMAGE books
- Enter text content for TEXTBOOKS
- Upload audio for chapters
- Replace media files when content changes

7.4 Content Visibility

Each book should have a status, such as:

- Draft
- Published
- Hidden
- Unpublished Changes (used when a published book has a pending background draft)

This allows authors to prepare content before release update live books seamlessly, and control visibility.

7.5 Version Control

The admin panel should automatically track content changes and update version numbers when:

- Text changes
- Images change
- Audio changes

This helps the mobile app know what has changed and what needs to be downloaded again.

8. Content Model

The content structure must be clear and consistent.

8.1 Author

Represents the person or organization owning a branded app.

8.2 Book

A book belongs to one author and contains:

- Title
- Description
- Cover image

- Type
- Status
- Version
- Display order
- Publication info

8.3 Chapter

A chapter belongs to one book and contains:

- Title
- Order index
- Version
- Media content
- Audio content if available

8.4 Chapter Resources

For IMAGE Books

A chapter may contain:

- Multiple page images
- Optional audio

For Textbooks

A chapter may contain:

- Written text
- Optional audio

9. Update and Synchronization Logic

The update system is one of the most important parts of the project.

9.1 Version Tracking

Each book and chapter should have version numbers. When content changes, the backend increments the version.

9.2 Update Check Process

The app checks a lightweight updates endpoint to compare:

- Local SQLite version
- Server version

If the server version is newer, the app knows an update exists.

9.3 Update Rules

Text Updates

If only text content changes:

- The app can sync the updated text automatically.
- The reader does not need to manually download anything.

Image Updates

If an image file changes:

- The app should alert the user that the chapter has been updated.
- The user can choose to download the new version.

Audio Updates

If an audio file changes:

- The app should alert the user.
- The user can choose to download the new audio version.

9.4 Why This Approach Is Useful

This method reduces unnecessary downloads and keeps the app responsive. It also makes it possible to update only the part of the chapter that changed instead of re-downloading the whole book.

10. Local Storage Strategy

SQLite is used for local app data only. It should not be treated as the main source of content truth.

10.1 Store in SQLite

- Book list metadata
- Chapter list metadata
- Reading progress
- Bookmarks
- Favourites
- Download status
- Local settings
- Local version numbers

10.2 Store as Files

Large assets should be stored as files on device storage, not as SQLite rows. This includes:

- Page images
- Audio files
- Downloaded media

10.3 Backend Storage

The backend and MySQL database should store:

- Author records
- Books
- Chapters
- Version numbers
- File references
- Publishing status

11. Backend Responsibilities

The backend must handle the following responsibilities:

- Authenticate admin users
- Verify author access
- Store and retrieve books, chapters, and media references
- Filter all data by authorId
- Return updates for version checks
- Manage publishing and hidden states
- Protect author content from cross-access
- Serve only the content that belongs to the correct author app

12. Database Schema Overview

Below is a high-level structure for the MySQL database.

12.1 authors

Stores author accounts and author-related metadata.

Example fields:

- id
- name
- email
- password_hash
- status
- role
- created_at
- updated_at

12.2 Super Admins

Controllers for all Authors create account for them maintain the system.

Example fields:

- Id
- author_id
- username or email
- password_hash (nullable for SSO users)
- oauth_provider (e.g., 'google', 'local')

- provider_id (Google user ID)
- role
- status
- created_at
- updated_at

12.3 books

Stores book-level information.

Example fields:

- id
- author_id
- parent_book_id (nullable, used to link a background draft to a live published book)
- title
- description
- cover_image
- type (IMAGE or TEXT)
- status (draft, published, unpublished_changes, hidden)
- version
- order_index
- publish_at
- created_at
- updated_at
- deleted_at (timestamp for soft deletes)

12.4 chapters

Stores chapter records.

Example fields:

- Id
- author_id
- book_id
- parent_chapter_id (nullable, used to link a draft chapter to a live published chapter)
- title
- order_index

- version
- created_at
- updated_at
- deleted_at (timestamp for soft deletes)

12.5 chapter_pages

Used for IMAGE books to store page images.

Example fields:

- Id
- author_id
- chapter_id
- image_path
- order_index
- version
- created_at
- updated_at

12.6 chapter_texts

Used for Textbooks to store chapter content.

Example fields:

- Id
- author_id
- chapter_id
- content
- version
- created_at
- updated_at

12.7 chapter_audio

Stores audio information for chapters.

Example fields:

- Id
- author_id

- chapter_id
- audio_path
- duration
- version
- created_at
- updated_at

12.8 book_versions or metadata tables

Optional tables may be used to track broader version history and sync metadata.

13. Content Workflow

13.1 Creating an Author App

1. SuperAdmin creates the author account in the admin panel.
2. System sends an automated invite email to the new Author with a secure link to set their initial password.
3. SuperAdmin configures author branding values.
4. SuperAdmin sets the author ID inside the Flutter template.
5. Build the app for that author.
6. Publish the app to users.
7. Author logs in using Email/Password or Google SSO.

13.2 Creating a Book

1. Author logs in.
2. Author creates a new book.
3. Author chooses book type: IMAGE or TEXT.
4. Author uploads cover image and description.
5. Author creates chapters.
6. Author adds images, text, or audio depending on the book type.
7. Author publishes the book.

13.3 Updating Content

1. Author edits chapter content in the admin panel.
2. Backend updates the version number.
3. App checks for updates.

4. If the change is text-only, the app syncs automatically.
5. If the change is image or audio, the app asks the reader to download the update.

14. Development and Build Workflow

The mobile app should be built from a shared Flutter template.

14.1 Shared Template

The template should contain:

- Shared UI components
- Reading system
- Audio system
- SQLite handling
- Download manager
- Update checker
- Auth and config structure

14.2 Author Configuration

Each build should use a configuration file or setup script containing:

- authorId
- appName
- packageName
- appIcon
- splash screen
- API base URL
- brand colors

14.3 Build Automation

A build script should help generate a new author app quickly. This reduces manual work and makes it easier to launch many author-branded apps from one codebase.

15. Security Considerations

The system should be designed with basic security in mind.

15.1 Access Control

- Authors should only access their own books and content.
- Admin authentication must be required for all content management.
- API requests must validate author identity.

15.2 Password Security

- Passwords must be stored as hashes.
- Plain text passwords must never be stored in the database.

15.3 Content Protection

The system should reduce accidental or unauthorized access to books, media, and admin credentials. Full copy protection is not guaranteed, but good access control should be enforced.

16. Non-Functional Requirements

16.1 Performance

- App should load quickly.
- Downloading should not freeze the UI.
- Reading should remain smooth even with large chapters.
- The admin panel upload UI must include visible progress bars, error messages, and auto-resume capabilities for large media files if the connection drops.
- The admin panel must provide explicit guidance, file limits, and recommended tools for media compression prior to upload.

16.2 Reliability

- The app must work offline for already downloaded content.
- Sync should fail gracefully if the network is unstable.
- Partial downloads should be recoverable.

16.3 Maintainability

- One shared codebase should be easy to update.
- Backend logic should be clear and modular.
- Versioning should reduce confusion when content changes.

16.4 Scalability

- The system should support multiple authors.
- The database should be structured to add future features without major redesign.

17. MVP Scope

The first version should focus on the essentials.

17.1 In Scope

- Separate branded Flutter apps for authors
- Single shared admin panel
- Author login accounts
- Multiple books per author
- IMAGE books
- TEXT books
- Chapter-by-chapter downloads
- Audio streaming
- Audio download
- Offline reading with SQLite
- Reading progress
- Bookmarks
- Favorites
- Update detection
- Smart sync for text updates
- Manual download prompt for image/audio updates
- Book publishing and hiding
- Branding management

17.2 Out of Scope for MVP

- Reader accounts
- Cloud sync
- Payments
- Advanced analytics
- Editions support
- OCR search inside image books
- Full text search across images
- Social features such as comments or ratings

18. Future Enhancements

These features are not required for the first version, but the system should be designed so they can be added later:

- Reader accounts
- Cloud backup and sync
- Scheduled publishing
- Push notification campaigns
- Search inside image books using OCR
- Payments or subscriptions
- Reading analytics
- Multiple languages
- Web reader version
- Notes and annotations
- Download-all mode for complete offline libraries

19. Key Assumptions

This documentation is based on the following assumptions:

1. Each author gets a separate mobile app.
2. One author app belongs to one author only.
3. A single author can have multiple books.
4. A single book must be either IMAGE or TEXT.

5. Each chapter may have audio.
6. The backend is shared across all author apps.
7. The backend filters all content by author.
8. SQLite is used only for local reader data.
9. The first book is available immediately in the app.
10. Other books are downloaded from the internet when available.

20. Conclusion

This project combines branded mobile publishing, offline reading, and centralized content management in one system. The strongest part of the idea is the separation between:

- a shared backend,
- a single admin panel,
- and many author-specific Flutter apps.

That structure keeps development manageable while still giving each author a distinct product. With a well-designed content model and version system, the project can start simple and grow into a much stronger platform later.

The next step after this documentation is to turn it into:

- a database design,
- an API specification,
- and a development roadmap.